EQUITY

Finacle J2EE Customization

Deployment Guide

Author: Kennedy Gatimu

Applies to: Fincore and CRM customizations across all subsidiaries

Date: April 2026

Table of Contents

1. What Is This System?

Before this system existed, deploying a Finacle J2EE customization file meant manually SSHing into servers, copying files by hand, and hoping nothing went wrong. There were no backups, no audit trail, and no way to tell what changed or when.

This system replaces that process with a **fully automated deployment pipeline**. You push your file change to Git — Azure DevOps does the rest.

- It detects exactly which files changed and which servers need updating
- It copies each file to the correct location on the correct Finacle server
- It creates a timestamped backup of every file it overwrites or deletes
- It logs every step — what was deployed, when, to which server

You never need to SSH into a Finacle server to deploy a file. You just push to Git.

Two Applications

Application	Repository Folder	Contains
Fincore	`fincore/`	Core banking customizations (XSL, JS, XML, properties)
CRM	`crm/`	CRM customizations (XSL, JS, XML, properties)

Each application has its own independent pipeline. A Fincore change triggers the Fincore pipeline; a CRM change triggers the CRM pipeline. They never interfere with each other.

2. The Three Environments

Think of environments as **stages a change must pass through** before it reaches bank customers.

```
Developer  -->  SIT  -->  PREPROD  -->  PROD
                 |           |           |
              Dev test      QA         Live
```

Environment	Git Branch	Who Uses It
SIT (System Integration Testing)	`develop`	Developers
PREPROD (Pre-Production)	`release`	QA team, business users
PROD (Production)	`main`	All bank customers

How Changes Move Between Environments

Each environment is a **completely independent Git branch** with its own folder structure, its own scripts, and its own pipeline.

A push to `develop` has zero effect on `release` or `main`.

When a change is tested and approved in SIT and needs to go to PREPROD, you **apply that same change again** on the `release` branch independently.

You do not merge from one branch to another and nothing is promoted automatically.

This is by design — each environment has a different server topology and different folder layouts. What works in one environment cannot be blindly carried to another.

The same applies when moving from PREPROD to PROD — you apply the change again on the `main` branch, with approval from the team lead.

Bank Topology Differs Per Environment

The same bank may be grouped differently depending on the environment, and the folder structure in the repo reflects this.

Bank	SIT	PREPROD	PROD
11 — South Sudan	Group server	Group server	Group server
43 — DRC	Group server	Own dedicated server	Group server
50 — Rwanda	Group server	Own dedicated server	Own dedicated server (Branch + FI)
54 — Kenya	Group server	Group server	Group server
55 — Tanzania	Group server	Own dedicated server	Own dedicated server
56 — Uganda	Group server	Own dedicated server	Own dedicated server (Branch + FI)
99 — Finserve	Group server	Group server	Not present in PROD

3. Repository Structure Per Environment

Because each environment has a different bank topology, the folder structure in the repository **is different on each branch**. When you switch branches you will see a different layout — this is expected.

Understanding ‘global/’ and ‘local/’

These two folder names appear throughout the repo and have a specific meaning:

- **global/** contains scripts that are common to the whole server — not specific to any one bank. On the server, these files live at the root of the `custom/` directory, shared by all banks on that server.
- **local/** contains scripts that are specific to one bank. On the server, these files live inside that bank's own subfolder, for example `.../custom/50/`.

This distinction exists because every Finacle server has two path types: a shared root path for common scripts and per-bank subfolders for bank-specific ones.

SIT Branch ('develop') — One Group, All Banks

In SIT, all banks share one physical server. Every bank sits inside a single `group/` folder.

```
fincore/
  custom/
    group/
      global/    : scripts common to all banks on the SIT server
      11/        : South Sudan specific files
      43/        : DRC specific files
      50/        : Rwanda specific files
      54/        : Kenya specific files
      55/        : Tanzania specific files
      56/        : Uganda specific files
      99/        : Finserve specific files

crm/
  customization/
    group/
      global/
      11/  43/  50/  54/  55/  56/  99/
```

`group/global/` holds scripts common to all banks on the SIT server. Any file placed there goes to the root `custom/` path on the server, available to all banks. Bank-numbered folders hold files specific to that bank only.

PREPROD Branch ('release') — Group + Individual Servers

In PREPROD, banks 11, 54, and 99 share one Group server (EQUAT). Banks 43, 50, 55, and 56 each have their own dedicated server. The repo splits to match.

```
fincore/
  custom/
    group/           : Group server — banks 11, 54, 99
      global/        : scripts common to all banks on the Group server
      11/            : South Sudan specific files
      54/            : Kenya specific files
      99/            : Finserve specific files
    43/              : DRC own server
      global/        : scripts common to the DRC server
      local/         : DRC bank-specific files
    50/              : Rwanda own server
      global/        : scripts common to the Rwanda server
      local/         : Rwanda bank-specific files
    55/              : Tanzania own server
      global/        : scripts common to the Tanzania server
      local/         : Tanzania bank-specific files
    56/              : Uganda own server
      global/        : scripts common to the Uganda server
      local/         : Uganda bank-specific files

crm/
  customization/
    (mirrors the same structure)
```

For the Group server, `group/global/` holds scripts common to all banks sharing that server (11, 54, 99). Each bank-numbered folder under `group/` holds files specific to that bank.

For the subsidiary servers (43, 50, 55, 56), `global/` holds scripts that go to the root `custom/` path on that server, and `local/` holds scripts that go to the bank's own subfolder on that server.

PROD Branch ('main') — Branch, FI, and Tanzania

Production introduces **Branch and FI servers**. The Group server and some subsidiaries each have two physical machines — one for Branch users and one for FI (Financial Institution) users. Tanzania is the exception and has a single server covering both.

The repo has three top-level folders under `fincore/` and `crm/`:

```
fincore/
  branch/
    group/          : Group Branch server (EQPROD) — banks 11, 43, 54
      global/       : scripts common to the Group Branch server
      11/           : South Sudan branch-specific files
      43/           : DRC branch-specific files
      54/           : Kenya branch-specific files
    50/             : Rwanda Branch own server
      global/       : scripts common to the Rwanda Branch server
      local/        : Rwanda branch-specific files
    56/             : Uganda Branch own server
      global/       : scripts common to the Uganda Branch server
      local/        : Uganda branch-specific files
  fi/
    group/          : Group FI server (EQPRODFI) — banks 11, 43, 54
      global/       : scripts common to the Group FI server
      11/           : South Sudan FI-specific files
      43/           : DRC FI-specific files
      54/           : Kenya FI-specific files
    50/             : Rwanda FI own server
      global/       : scripts common to the Rwanda FI server
      local/        : Rwanda FI-specific files
    56/             : Uganda FI own server
      global/       : scripts common to the Uganda FI server
      local/        : Uganda FI-specific files
  tanzania/
    55/             : Tanzania single server — covers both Branch and FI
      global/       : scripts common to the Tanzania server
      local/        : Tanzania bank-specific files

crm/
  customization/
    (mirrors the same branch/fi/tanzania structure)
```

If your change applies to both Branch and FI users, you update the file under both `branch/` and `fi/` — they are separate files going to two separate physical machines.

> *DRC (43) in PROD: DRC currently sits on the Group server ('branch/group/43/' and 'fi/group/43/'). When DRC's own dedicated servers are provisioned, the deployment script will be updated — the logic is already prepared and waiting.*

4. Where Does My File Go? (Path Conventions)

The folder structure in the repo mirrors the folder structure on the Finacle server. The pipeline reads the file's location in the repo and automatically determines where to place it on the server. You do not configure this — the path is the configuration.

SIT Path Examples

Repo path	Server path
`fincore/custom/group/54/cif/display/Account.xsl`	`/equity_fe/EQSIT/FrontEnd/FinacleApps/finbranch.war/custom/54/cif/display/Account.xsl`
`fincore/custom/group/global/cif/js/custom.js`	`/equity_fe/EQSIT/FrontEnd/FinacleApps/finbranch.war/custom/cif/js/custom.js`
`crm/customization/group/43/cif/CustomerView.xsl`	`/equity_fe/EQSIT/FrontEnd/FinacleApps/FinacleCRM.ear/FinacleCRM.war/Customization/43/cif/CustomerView.xsl`

PREPROD Path Examples

Group server (banks 11, 54, 99 — EQUAT):

Repo path	Server path
`fincore/custom/group/54/cif/display/Account.xsl`	`/equity_fe/EQUAT/.../finbranch.war/custom/54/cif/display/Account.xsl`
`fincore/custom/group/global/cif/js/custom.js`	`/equity_fe/EQUAT/.../finbranch.war/custom/cif/js/custom.js`
`crm/customization/group/99/cif/CustomerView.xsl`	`/equity_fe/EQUAT/.../FinacleCRM.war/Customization/99/cif/CustomerView.xsl`

Subsidiary own servers (banks 43, 50, 56):

Repo path	Server path
`fincore/custom/43/local/cif/display/Account.xsl`	`/u01/equity_fe/FinacleApps/finbranch.war/custom/43/cif/display/Account.xsl`
`fincore/custom/50/global/cif/js/custom.js`	`/u01/equity_fe/FinacleApps/finbranch.war/custom/cif/js/custom.js`
`crm/customization/56/local/cif/CustomerView.xsl`	`/u01/equity_fe/FinacleApps/FinacleCRM.war/Customization/56/cif/CustomerView.xsl`

Tanzania (bank 55):

Repo path	Server path
`fincore/custom/55/local/cif/display/Account.xsl`	`/finacle/EQTZPPROD/Fin10218/J2EE/Deployment/finbranch.ear/finbranch.war/custom/55/cif/display/Account.xsl`
`fincore/custom/55/global/cif/js/custom.js`	`/finacle/EQTZPPROD/Fin10218/J2EE/Deployment/finbranch.ear/finbranch.war/custom/cif/js/custom.js`

PROD Path Examples

Group Branch server (EQPROD) and Group FI server (EQPRODFI):

Repo path	Server path
fincore/branch/group/54/cif/display/Account.xsl	/equity_fe/EQPROD/FrontEnd/FinacleApps/finbranch.war/custom/54/cif/display/Account.xsl
fincore/branch/group/global/cif/js/custom.js	/equity_fe/EQPROD/FrontEnd/FinacleApps/finbranch.war/custom/cif/js/custom.js
fincore/fi/group/54/cif/display/Account.xsl	/equity_fe/EQPRODFI/FrontEnd/FinacleApps/finbranch.war/custom/54/cif/display/Account.xsl
fincore/fi/group/global/cif/js/custom.js	/equity_fe/EQPRODFI/FrontEnd/FinacleApps/finbranch.war/custom/cif/js/custom.js

Rwanda and Uganda Branch and FI (banks 50, 56):

Repo path	Server path
fincore/branch/50/local/cif/display/Account.xsl	/u01/equity_fe/FinacleApps/finbranch.war/custom/50/cif/display/Account.xsl
fincore/fi/56/global/cif/js/custom.js	/u01/equity_fe/FinacleApps/finbranch.war/custom/cif/js/custom.js

> *Rwanda and Uganda Branch and FI servers write to the same server path — what changes is which physical machine is used.*

Tanzania (single server — bank 55):

Repo path	Server path
fincore/tanzania/55/local/cif/display/Account.xsl	/finacle/EQTZPROD/Fin10218/J2EE/Deployment/finbranch.ear/finbranch.war/custom/55/cif/display/Account.xsl
fincore/tanzania/55/global/cif/js/custom.js	/finacle/EQTZPROD/Fin10218/J2EE/Deployment/finbranch.ear/finbranch.war/custom/cif/js/custom.js

5. Git Basics — Start Here If You Are New to Git

What Is Git?

Git is a version control system — a tool that tracks every change ever made to every file in the repository. It remembers who changed what, when, and why. It also lets multiple developers work on the same codebase at the same time without overwriting each other's work.

What Is a Branch?

A branch is an independent copy of the codebase. Changes on one branch do not affect other branches. Think of it as three separate desks — what you put on the SIT desk stays there and does not appear on the PREPROD desk.

```
develop  -->  SIT environment
release  -->  PREPROD environment
main     -->  PROD environment
```

Essential Git Commands

Check your current branch:

```
git branch
```

The branch marked with `*` is your active branch. Always confirm this before making any changes.

Switch to a branch:

```
git checkout develop    # SIT
git checkout release    # PREPROD
git checkout main       # PROD
```

Pull the latest code from Azure DevOps:

```
git pull origin develop    # pull latest SIT code
git pull origin release    # pull latest PREPROD code
git pull origin main       # pull latest PROD code
```

> *Always use 'git pull origin <branch>' — not just 'git pull'. The short form 'git pull' relies on local Git configuration to know which branch to pull from. If that configuration is wrong, or you are on the wrong branch, you may pull from somewhere unintended — silently. The explicit form 'git pull origin <branch>' always does exactly what you typed, regardless of any configuration. Make this a habit.*

Check what files you have changed:

```
git status
```

See exactly what changed inside a file:

```
git diff fincore/custom/group/54/cif/display/Account.xsl
```

Lines beginning with `+` are additions. Lines beginning with `-` are removals.

Stage a file (mark it for the next commit):

```
git add fincore/custom/group/54/cif/display/Account.xsl
```

Commit with a message:

```
git commit -m "Update Account.xsl for Kenya (54) — fix balance rounding CHG0012345"
```

A good commit message states what changed, which bank it affects, and the change request number.

Push your working branch to Azure DevOps:

```
git push origin CHG0012345
```

> *You cannot push directly to 'develop', 'release', or 'main'. These three branches are protected. Azure DevOps will reject any direct push to them. Instead, you push your own working branch and then open a Pull Request to merge it into the target branch. The full workflow is covered in Section 7.*

The Golden Rules

1. **Always 'git pull origin <branch>' before you start working.** This is the single most important habit — see Section 6 for why.
2. **Always confirm your branch before making any changes.** Two seconds with `git branch` can save hours of cleanup.
3. **One logical change per commit.** Do not bundle unrelated changes into one commit.
4. **Write meaningful commit messages.** "Updated file" tells nobody anything. "Fix balance display rounding for Kenya (54) — CHG0012345" tells everything.
5. **Never push to 'main' without team lead approval.** Production deployments are irreversible without a rollback.

6. Understanding and Resolving Merge Conflicts

What Is a Merge Conflict?

A merge conflict happens when **two developers edit the same file at the same time** and Git cannot automatically decide which version to keep.

Here is a practical example. You check out the `release` branch on Monday morning and start editing `Account.xsl`. While you are working, a colleague also edits the same file and pushes their change first. When you try to push, Git sees that the file on the remote is now different from the version you started with — and you both changed it. Git cannot merge them automatically and reports a conflict.

Why Pulling First Prevents Most Conflicts

If you had run `git pull origin release` **before** starting your edit, you would have received your colleague's latest version first. You would then be working on the already-updated file, and when you push, there is nothing to conflict with.

This is why pulling before every work session is the most important Git habit. It does not guarantee you will never get a conflict — someone could push while you are mid-edit — but it eliminates the vast majority of them.

How to Recognise a Conflict

When a conflict occurs, Git tells you:

```
CONFLICT (content): Merge conflict in
fincore/custom/group/54/cif/display/Account.xsl
Automatic merge failed; fix conflicts and then commit the result.
```

Running `git status` will show the conflicting file marked as `both modified`.

What a Conflict Looks Like Inside the File

Git inserts markers directly into the file to show you both versions:

```
<<<<<<< HEAD
<xsl:value-of select="format-number(Balance, '#,##0.00')"/>
=======
<xsl:value-of select="format-number(Balance, '#,##0.000')"/>
>>>>>>> release
```

- Everything between `<<<<<<< HEAD` and `=======` is your version
- Everything between `=======` and `>>>>>>> release` is the incoming version from the remote
- Git is asking you: which one should survive?

How to Resolve a Conflict — Step by Step

Step 1 — Open the conflicting file in your editor. Search for `<<<<<<<` to find every conflict block.

Step 2 — Decide which version to keep. You have three choices: keep your version, keep the incoming version, or write a combined version that takes the best of both.

Step 3 — Remove all conflict markers. The final file must contain none of `<<<<<<<`, `=======`, or `>>>>>>>`. It should look like a normal, clean file.

For example, if you decide the incoming version is correct, the block above becomes:

```
<xsl:value-of select="format-number(Balance, '#,##0.000')"/>
```

Step 4 — Stage and commit the resolved file:

```
git add fincore/custom/group/54/cif/display/Account.xsl
git commit -m "Resolve merge conflict in Account.xsl — kept three decimal format
CHG0012345"
```

Step 5 — Push:

```
git push origin release
```

> *Not sure which version to keep? Do not guess on a financial calculation or display file. Contact the colleague whose change caused the conflict, compare intentions, and agree together. Then one of you resolves and commits.*

If the Conflict Feels Overwhelming

Ask a senior developer. Merge conflicts are routine and there is no shame in asking for help. A bad resolution in a banking customization file is far worse than a short conversation.

7. Making a Deployment — Step by Step

How the Branch Policy Works

The branches `develop`, `release`, and `main` are **protected**. You cannot push directly to them — Azure DevOps will reject the attempt. This is a deliberate security control to ensure that every change is reviewed before it reaches any environment.

The correct workflow is:

6. Create your own working branch off the target branch
7. Make your changes on that working branch
8. Push your working branch to Azure DevOps
9. Open a Pull Request to merge your branch into the target branch
10. A reviewer approves the Pull Request
11. The Pull Request is merged — and the pipeline triggers automatically on the target branch

---

Branch Naming Convention

Name your working branch after the change request number. That is all you need.

```
CHG0012345
```

If no change request number exists yet, use a short description of what you are doing:

```
fix-kenya-account-xsl
```

Avoid vague names like `my-branch`, `test`, or `update`. Anyone looking at the branch list should immediately understand what it is about.

How to Create a Pull Request in Azure DevOps

Once you have pushed your working branch, open a Pull Request like this:

12. Go to **Repos > Pull Requests** in Azure DevOps
13. Click **New pull request**
14. Set the **source branch** to your working branch (e.g. `sit/CHG0012345-kenya-account-xsl`)
15. Set the **target branch** to the environment branch (`develop`, `release`, or `main`)
16. Give it a clear title — e.g. `Update Account.xsl for Kenya (54) — CHG0012345`
17. Add a short description of what changed and why
18. Add the relevant reviewers
19. Click **Create**

Once the Pull Request is approved, click **Complete** to merge it. The pipeline will trigger automatically on the target branch as soon as the merge is done.

Deploying to SIT ('develop' branch)

1. Switch to the SIT branch and pull the latest code

```
git checkout develop
git pull origin develop
```

2. Create your working branch

```
git checkout -b CHG0012345
```

3. Place or edit your file — all banks in SIT go under `fincore/custom/group/<bank_id>/` or `crm/customization/group/<bank_id>/`

4. Confirm only your intended files appear

```
git status
git diff fincore/custom/group/54/cif/display/Account.xsl
```

5. Stage and commit

```
git add fincore/custom/group/54/cif/display/Account.xsl
git commit -m "Test Account.xsl balance fix for Kenya (54) — CHG0012345"
```

6. Push your working branch

```
git push origin CHG0012345
```

7. Open a Pull Request in Azure DevOps — go to **Repos > Pull Requests**, click **New pull request**, set the source to `CHG0012345` and the target to `develop`. Give it a clear title, add a short description, and assign a peer or senior developer as the reviewer. Once approved, click **Complete** to merge. The SIT pipeline triggers automatically.

Deploying to PREPROD ('release' branch)

Only raise a PR to `release` after the change has been tested and signed off in SIT.

1. Switch to the PREPROD branch and pull the latest code

```
git checkout release
git pull origin release
```

2. Create your working branch

```
git checkout -b CHG0012345
```

3. Place or edit your file in the correct PREPROD folder (see Sections 3 and 4)

- Banks 11, 54, 99 — `fincore/custom/group/<bank_id>/`
- Banks 43, 50, 55, 56 — `fincore/custom/<bank_id>/local/` or `global/`

4. Confirm only your intended files appear

```
git status
git diff fincore/custom/group/54/cif/display/Account.xsl
```

5. Stage and commit

```
git add fincore/custom/group/54/cif/display/Account.xsl
git commit -m "Update Account.xsl for Kenya (54) — fix balance rounding CHG0012345"
```

6. Push your working branch

```
git push origin CHG0012345
```

7. Open a Pull Request in Azure DevOps — go to **Repos > Pull Requests**, click **New pull request**, set the source to `CHG0012345` and the target to `release`. Include the SIT sign-off reference in the description, and add the team lead and a senior developer as reviewers. Once approved, click **Complete** to merge. The PREPROD pipeline triggers automatically.

Deploying to PROD ('main' branch)

Only raise a PR to `main` after PREPROD sign-off and explicit approval from the team lead.

> *Important: Merging into 'main' deploys directly to live banking servers. Verify everything carefully before raising this PR.*

1. Switch to the PROD branch and pull the latest code

```
git checkout main
git pull origin main
```

2. Create your working branch

```
git checkout -b CHG0012345
```

3. Place or edit your file in the correct PROD folder (see Sections 3 and 4)

- Branch users — `fincore/branch/group/<bank_id>/` or `fincore/branch/<bank_id>/local/`
- FI users — `fincore/fi/group/<bank_id>/` or `fincore/fi/<bank_id>/local/`
- Tanzania — `fincore/tanzania/55/local/` or `fincore/tanzania/55/global/`
- If the change applies to both Branch and FI users, update files under both `branch/` and `fi/`

4. Confirm your changes carefully

```
git status
git diff fincore/branch/group/54/cif/display/Account.xsl
```

5. Stage and commit

```
git add fincore/branch/group/54/cif/display/Account.xsl
git add fincore/fi/group/54/cif/display/Account.xsl
git commit -m "Deploy Account.xsl fix for Kenya (54) — CHG0012345 PREPROD approved"
```

6. Push your working branch

```
git push origin CHG0012345
```

7. Open a Pull Request in Azure DevOps — go to **Repos > Pull Requests**, click **New pull request**, set the source to `CHG0012345` and the target to `main`. Reference the approved change request and confirm PREPROD sign-off in the description. The team lead is the required reviewer for all PROD pull requests. Once approved, click **Complete** to merge. The PROD pipeline triggers automatically.

8. Monitoring Your Deployment in Azure DevOps

After you push, Azure DevOps runs two things in sequence: a **Build** (CI) that packages your files, followed by a **Release** (CD) that deploys them to the server.

Step 1 — Watch the Build

20. Open Azure DevOps and go to **Pipelines > Pipelines**
21. Click the pipeline for your app:

- Fincore changes: `j2ee-fincore-uat-scripts CI`
- CRM changes: `j2ee-crm-uat-scripts CI`

22. A new build will appear at the top with a spinning icon while it runs. Click it.
23. You will see a list of steps. Click any step to open its log.

What a healthy build looks like:

Every step has a green checkmark. In the **Check Modified Files** step you will see a summary of what was detected:

```
[INFO] New files      : 1
[INFO] Modified files : 0
[INFO] Deleted files  : 0
```

Further down you will see which servers were flagged for deployment:

```
DEPLOY GROUP  (banks 11, 54, 99 - EQUAT server)  : true
DEPLOY 43     (DRC)                               : false
DEPLOY 50     (Rwanda)                            : false
```

This tells you your change affected the Group server only — DRC and Rwanda were not touched.

If a build step turns red, click it and read the log from the bottom up — the actual error is usually near the end.

Step 2 — Watch the Release

Once the build goes green, a Release is triggered automatically.

24. Go to **Pipelines > Releases**
25. Find `j2ee-fincore-uat-scripts CD` or `j2ee-crm-uat-scripts CD`
26. A new release will appear (e.g. Release-7). Click it.
27. You will see all the deployment stages. Only stages for servers with changed files will run.

Stage colours:

- **Green (Succeeded)** — Files deployed successfully to that server
- **Red (Failed)** — Something went wrong. Click the stage to investigate.
- **Grey (Not triggered)** — This server had no changes — correctly skipped

Step 3 — Read the Deployment Log

Click a stage > click the **SSH** task inside it > click **Logs**.

Scroll to the very bottom and look for the summary block:

```
============================================================
              DEPLOYMENT EXECUTION SUMMARY
============================================================
ENVIRONMENT  : PREPROD
TARGET       : group
NEW FILES DEPLOYED      : 0
MODIFIED FILES DEPLOYED : 1
DELETED FILES DEPLOYED  : 0
SUCCESSFUL DEPLOYMENTS  : 1
FAILED DEPLOYMENTS      : 0
STATUS       : SUCCESS
============================================================
```

'STATUS : SUCCESS' — deployment completed. You are done.

'FAILED DEPLOYMENTS : 1' — something went wrong. Scroll up in the same log and look for a line starting with `[ERROR]`. That line will tell you exactly what failed and why. Common errors are listed in Section 12.

9. What Happens on the Finacle Server

Staging and Work Directories

When a deployment starts, two temporary directories are created on the Finacle server, both named after the Release ID:

```
/tmp/j2ee-deploy-5198/      : staging directory owned by the CD agent
  ├── j2ee-scripts.tar.gz   : archive containing your changed files
  ├── deploy-j2ee-uat.sh    : deployment script
  └── suScript.sh           : entry point

/tmp/j2ee-work-5198/        : work directory owned by the Finacle application user
  ├── newFiles/
  ├── modifiedFiles/
  ├── deletedFiles/
  └── deployment-manifest.txt
```

Both directories are deleted automatically at the end of deployment.

What the Deployment Script Does

For every file listed in the deployment manifest:

New files — Creates any missing parent directories on the server, then copies the file to its destination.

Modified files — Creates a timestamped backup of the existing file first, then copies the new version:

```
/equity_fe/.../custom/54/backup/cif/display/Account.xsl_20260417_080312
```

Deleted files — Creates a timestamped backup, then removes the file from the server.

The `backup/` folder is your safety net. Nothing is ever overwritten or deleted without a copy being saved first.

10. Deployment Backups

Before the deployment script overwrites or deletes any file on the Finacle server, it automatically saves a copy of the original. You do not need to do anything — this happens on every deployment without exception. New files being added for the first time are not backed up since there is nothing to back up yet.

Change Type	Backup Taken?
New file	No
Modified file	Yes — original saved before overwrite
Deleted file	Yes — saved before deletion

Where Backups Live

Every backup is stored in a `backup/` folder on the server, right next to where the original file lives. The backup file has the same name as the original, with the date and time of the deployment appended to it.

For example, if `Account.xsl` for Kenya (bank 54) was overwritten on 17 April 2026 at 08:03, the backup is saved as:

```
.../custom/54/backup/cif/display/Account.xsl_20260417_080312
```

And if a shared global file like `custom.js` was overwritten at the same time, its backup is saved as:

```
.../custom/backup/cif/js/custom.js_20260417_080312
```

The same applies to CRM files and files across all environments — SIT, PREPROD, and PROD.

To restore from a backup, use Option 1 in Section 11 — redeploying a previous release is the quickest recovery path and does not require manual file handling.

11. How to Roll Back a Deployment

Option 1 — Redeploy a Previous Release (Recommended)

The simplest rollback. It re-runs an earlier Azure DevOps release using the files that were deployed at that time.

28. Go to **Pipelines > Releases**
29. Open the CD pipeline for the affected app
30. Find the release with the last known-good version (e.g. Release-4)
31. Click it, find the affected server stage, then click **Deploy**
32. Confirm — the pipeline re-runs with Release-4's files

> *Note: This re-deploys all files from that release. It does not remove files added in a later release. For that case, contact the team lead.*

Option 2 — Restore From Server Backup

Every overwritten or deleted file has a timestamped backup on the server (see Section 10). If a specific file needs to be restored without redeploying the full release, a support engineer can locate the correct backup by its timestamp and copy it back to the original location. Use Option 1 first — this is a manual fallback.

12. Common Errors and What They Mean

CI Pipeline

"No manifest file found" or "Manifest file is empty"

No J2EE files were detected in your commit. Check that you are on the correct branch, your file is in the correct folder (`fincore/` or `crm/`), and the pipeline is for the correct app.

"Missing or invalid --app argument"

Internal pipeline configuration error. Contact the DevOps team.

CD Pipeline

"Manifest file not found: deployment-manifest.txt"

The artifact was packaged but the manifest is missing. Check the CI build logs — if zero files were detected, the artifact is empty.

"Unknown bank_id '...' for group target"

A file is in the wrong folder. For example, `custom/group/43/` on the `release` branch — bank 43 in PREPROD has its own server and belongs in `custom/43/local/`. Refer to Section 3 and Section 4.

"Backup failed for: ..."

The script could not create a backup before overwriting a file. Nothing is deployed when a backup fails. Contact the infrastructure team.

"chmod failed" or "Extraction failed"

A server-side permission issue. Contact the infrastructure team.

Git Errors

"Your local branch is behind the remote"

Someone pushed while you were working. Run `git pull origin <branch>`, then push again. If there is a conflict, see Section 6.

"Merge conflict"

Two developers edited the same file. Follow the steps in Section 6 to resolve.

"unable to push — protected branch"

You tried to push directly to `develop`, `release`, or `main`. These branches are protected. Push to your own working branch instead, then open a Pull Request. See Section 7.

"Permission denied (publickey)"

Your SSH key is not set up for the Azure DevOps repository. Contact the DevOps team.

13. Server Reference — SIT, PREPROD, PROD

Bank IDs

Bank ID	Country	SIT	PREPROD	PROD
11	South Sudan	Group	Group	Group
43	DRC	Group	Own server	Group (own server coming)
50	Rwanda	Group	Own server	Own server (Branch + FI)
54	Kenya	Group	Group	Group
55	Tanzania	Group	Own server	Own server
56	Uganda	Group	Own server	Own server (Branch + FI)
99	Finserve	Group	Group	Not present

SIT Servers

Server	Banks	Fincore Base Path	CRM Base Path
EQSIT	All banks	`/equity_fe/EQSIT/FrontEnd/FinacleApps/finbranch.war/custom/`	`/equity_fe/EQSIT/FrontEnd/FinacleApps/FinacleCRM.ear/FinacleCRM.war/Customization/`

PREPROD Servers

Server	Banks	Fincore Base Path	CRM Base Path
EQUAT (Group)	11, 54, 99	`/equity_fe/EQUAT/FrontEnd/FinacleApps/finbranch.war/custom/`	`/equity_fe/EQUAT/FrontEnd/FinacleApps/FinacleCRM.ear/FinacleCRM.war/Customization/`
DRC server	43	`/u01/equity_fe/FinacleApps/finbranch.war/custom/`	`/u01/equity_fe/FinacleApps/FinacleCRM.ear/FinacleCRM.war/Customization/`
Rwanda server	50	`/u01/equity_fe/FinacleApps/finbranch.war/custom/`	`/u01/equity_fe/FinacleApps/FinacleCRM.ear/FinacleCRM.war/Customization/`
Tanzania (EQTZPPROD)	55	`/finacle/EQTZPPROD/Fin10218/J2EE/Deployment/finbranch.ear/finbranch.war/custom/`	`/finacle/EQTZPPROD/Fin10218/J2EE/Deployment/FinacleCRM.ear/FinacleCRM.war/Customization/`
Uganda server	56	`/u01/equity_fe/FinacleApps/finbranch.war/custom/`	`/u01/equity_fe/FinacleApps/FinacleCRM.ear/FinacleCRM.war/Customization/`

PROD Servers

Server	Type	Banks	Fincore Base Path
EQPROD (Group Branch)	branch	11, 43, 54	`/equity_fe/EQPROD/FrontEnd/FinacleApps/finbranch.war/custom/`
EQPRODFI (Group FI)	fi	11, 43, 54	`/equity_fe/EQPRODFI/FrontEnd/FinacleApps/finbranch.war/custom/`
Rwanda Branch	branch	50	`/u01/equity_fe/FinacleApps/finbranch.war/custom/`
Rwanda FI	fi	50	`/u01/equity_fe/FinacleApps/finbranch.war/custom/`
Uganda Branch	branch	56	`/u01/equity_fe/FinacleApps/finbranch.war/custom/`
Uganda FI	fi	56	`/u01/equity_fe/FinacleApps/finbranch.war/custom/`
Tanzania (EQTZPROD)	—	55	`/finacle/EQTZPROD/Fin10218/J2EE/Deployment/finbranch.ear/finbranch.war/custom/`

> *Rwanda and Uganda Branch/FI servers share the same file system path — the pipeline SSHs into different physical machines.*

For CRM PROD paths, replace `finbranch.war/custom/` with `FinacleCRM.ear/FinacleCRM.war/Customization/` in all entries above.

14. Quick Reference Card

Branch and Environment

Environment	Branch	Push command
SIT	`develop`	`git push origin develop`
PREPROD	`release`	`git push origin release`
PROD	`main`	`git push origin main`

Repo Folder by Environment and Bank

Environment	Bank	Fincore folder
SIT — any bank	11, 43, 50, 54, 55, 56, 99	`fincore/custom/group/<bank_id>/`
SIT — global	—	`fincore/custom/group/global/`
PREPROD — Group banks	11, 54, 99	`fincore/custom/group/<bank_id>/`
PREPROD — Group global	—	`fincore/custom/group/global/`
PREPROD — subsidiary, bank-specific	43, 50, 55, 56	`fincore/custom/<bank_id>/local/`
PREPROD — subsidiary, common scripts	43, 50, 55, 56	`fincore/custom/<bank_id>/global/`
PROD — Group Branch	11, 43, 54	`fincore/branch/group/<bank_id>/`
PROD — Group FI	11, 43, 54	`fincore/fi/group/<bank_id>/`
PROD — subsidiary Branch	50, 56	`fincore/branch/<bank_id>/local/`
PROD — subsidiary FI	50, 56	`fincore/fi/<bank_id>/local/`
PROD — Tanzania	55	`fincore/tanzania/55/local/`

Deployment Checklist

```
  1.  git checkout <env-branch>                 develop | release | main
  2.  git pull origin <env-branch>              always pull before you start
  3.  git checkout -b CHG0012345              create your working branch
  4.  Place or edit your file                   see table above for correct folder
  5.  git status                                confirm only intended files changed
  6.  git diff <file>                           review changes line by line
  7.  git add <file>
  8.  git commit -m "message — CHGxxxxxxx"
  9.  git push origin <your-working-branch>     push your branch (not the env
branch)
  10. Azure DevOps > Repos > Pull Requests      open a PR to develop | release |
main
  11. PR approved and merged                    pipeline triggers automatically
  12. Azure DevOps > Pipelines                  watch CI build go green
  13. Azure DevOps > Releases                   watch CD stages go green
  14. Deployment log                            confirm STATUS : SUCCESS
```

Need Help?

Issue	Who to Contact
File landed in wrong place on server	DevOps team
Pipeline not triggering after push	DevOps team
Permission error on server	Infrastructure team
Git merge conflict	Section 6 first, then senior developer
Urgent rollback needed	Team lead — use Section 11, Option 1